

Review

2.6 Syllabus covered

- ✓ Binary numbers
- ✓ Generate minterms, maxterms, SOP canonical form and POS canonical forms and convert between them
- ✓ Understand and use the laws and theorems of Boolean Algebra
- ✓ Perform algebraic simplification using Boolean algebra
- ✓ Simplification using K-maps
- ✓ Derive sum of product and product of sums expressions for a combinational circuit
- ✓ Convert combinational logic to NAND-NAND and NOR-NOR forms
- Hexadecimal, Sign-magnitude, One's-complement and Two's complement. Conversions between them.
- Design combinational circuits for positive and negative logic
- Design Hazard-free two level circuits and understand Hazards in multi-level circuits
- Compute noise margin of one device
- Describe how tri-state and open-collector outputs are different from totem-pole outputs.
- Different between and limitations of master-slave and edge-triggered flip-flops.
- Compute fan out and noise margin of one device driving the same time
- Know the differences and similarities between PAL, PLA, and ROMs and can use each for logic design
- Design combinational circuits using multiplexers and decoders
- Analyze a sequential circuit and derive a state-table and a state-graph
- Understand the difference between synchronous and asynchronous inputs
- Derive a state graph or state table from a word description of the problem
- Reduce the number of states in a state table using row reduction and implication tables
- Perform a state assignment using the guideline method

- Implement a design using JK, SR, D or T flip-flops
- Analyse and design both Mealy and Moore sequential circuits with multiple inputs and multiple outputs
- Convert between Mealy and Moore designs
- Partition a system into multiple state machines

2.6.1 Labs (not questioned in exams)

- Use computer tools to enter designs graphically and HDL
- Simulate designs using computer tools
- Use computer tools to program gate arrays logic and debug and test

Chapter 3

Sample midterm exam

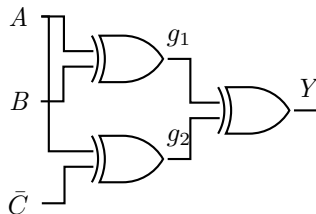
Student Name:

Student Email:

3.1 Instructions

- Time allowed is 50 minutes. (This sample exam might be lengthier than the actual exam. Same instruction will apply to the midterm.)
- In order to minimize distraction to your fellow students, you may not leave during the last 10 minutes of the examination.
- The examination is closed-book. One 8x11in cheatsheet is allowed.
- Non-programmable calculators are permitted.
- The maximum number of marks is 80, as indicated; the midterm examination amounts 10% toward the final grade.
- Please colored pen/pencils for K-maps, use a pen or heavy pencil to ensure legibility.
- Please show your work; where appropriate, marks will be awarded for proper and well-reasoned explanations.

Problem 3.1. Consider the circuit below

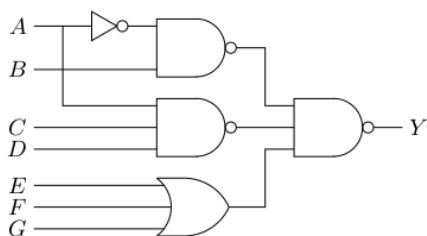


By algebraic manipulation, prove or disprove that $Y = \bar{B}\bar{C} + BC$ (10 marks).

Problem 3.2. Use the following 4-variable K-map for $F(A, B, C, D)$, and find a minimal sum of products (SOP) expression for F (15 marks)

		AB			
		00	01	11	10
CD	00	1	0	0	1
	01	1	1	0	1
	11	0	1	0	0
	10	0	1	1	0

Problem 3.3. Use boolean algebra to find an SOP expression for Y in the circuit below. Draw an equivalent circuit beside the given circuit (5 marks).



Problem 3.4. Consider the function Y given below.

$$Y(A, B, C, D) = \sum m(0, 3, 5, 7, 8, 14) + d(2, 12, 15)$$

1. Draw a K-maps to derive a minimum SOP and POS expressions for Y . Indicate all essential prime implicants for Y or $\bar{Y}$ in your K-maps (20 marks).
2. Sketch a two-level NOR-NOR circuit for Y . Assume that A , B , C , and D are available in true and complimentary forms (5 marks).
3. Write Y in Product of sums (POS) canonical form (5 marks).

Problem 3.5. Design a minimal SOP circuit to add two two-bit unsigned numbers. Denote the two bits of first number as A_1A_0 and the two bits of second number as B_1B_0 . The result will be a 2-bit sum S_1S_0 and a carry C . Start with filling out the following truth table (3 example rows are provided) and then use K-maps to find minimal SOP for S_1 , S_0 and a single carry bit C_1 (20 marks).

A_1	A_0	B_1	B_0	C_1	S_1	S_0
0	0	0	0			
0	0	0	1			
0	0	1	0			
0	0	1	1			
0	1	0	0			
0	1	0	1	0	1	0
0	1	1	0			
0	1	1	1			
1	0	0	0			
1	0	0	1			
1	0	1	0			
1	0	1	1			
1	1	0	0			
1	1	0	1	1	0	0
1	1	1	0			
1	1	1	1	1	1	0

Chapter 4

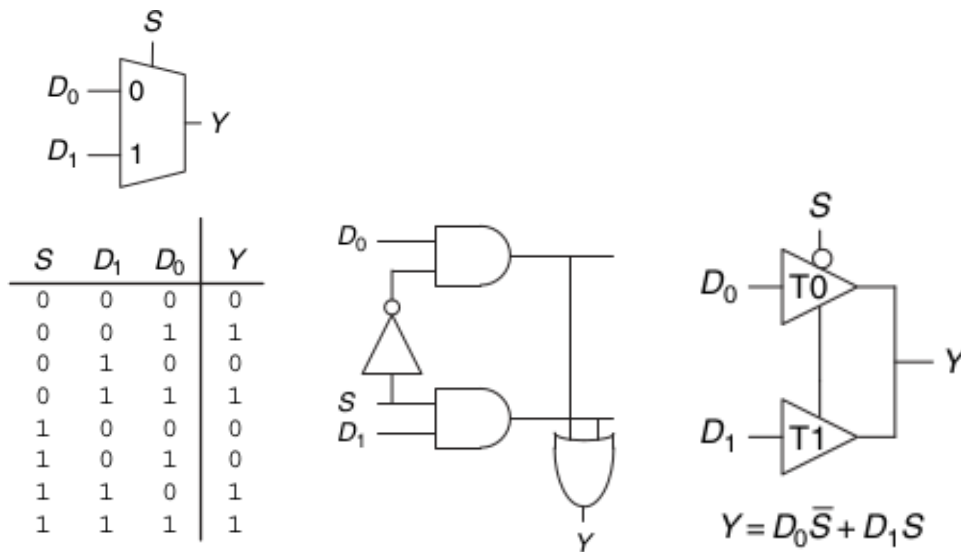
Muxes and Decoders

4.1 Objectives

1. Design combinational circuits using multiplexers and decoders

4.2 Design combinational circuit using multiplexers [1, Section 2.8.1]

4.2.1 Review: 2to1 Multiplexer (MUX)



4.2.2 Wider multiplexers

Draw the symbol for a 4:1 MUX, an 8:1 MUX and a $2^N : 1$ MUX and write corresponding Boolean expressions.

Example 4.1. Design a circuit for $Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$ using a 8:1 MUX.

Remark 4.1. A $2^N : 1$ MUX can be used to program any N -input logic function.

Example 4.2. Design a circuit for $Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$ using a 4:1 MUX and NOT gates only.

Remark 4.2. A $2^{N-1} : 1$ MUX can be used to program any N -input logic function, if we use literals on the input side.

Example 4.3. Design a circuit for $Y = \bar{A}C + \bar{A}B + B\bar{D}$ using a 8:1 MUX and NOT gates only. Also design using 4:1 MUX and other gates. fewest gates.

Problem 4.1. (15 marks [1, Ex-2.42]) Implement the function $Y = BC + \bar{A}\bar{B}\bar{C} + B\bar{C}$ using

1. an 8:1 multiplexer
2. a 4:1 multiplexer and no other gates
3. a 2:1 multiplexer, one OR gate, and an inverter

4.3 Encoders and Decoders

Example 4.4. Draw the symbol and the truth table for 2:4 decoder. Also write the logic expressions.

Example 4.5. Draw the symbol and the truth table for 3:8 decoder, 4:16 decoder and $N : 2^N$ decoder. Also write the logic expressions.

Example 4.6. Design a circuit for a XOR gate using a 2:4 decoder and an OR gate.

Example 4.7. Design a circuit for $Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$ using a 3:8 decoder and an OR gate.

4.3.1 (Priority) Encoders

Example 4.8. Draw symbol and truth table for a 4:2 priority encoder.

Example 4.9. Draw symbol and truth table for a 8:3 priority encoder.